

《软件过程与管理》第一次作业

项目名称：校园二手交易微信小程序开发项目

团队规模：5 人（全栈开发 2 人，前端 1 人，产品兼测试 1 人，UI 兼运营 1 人）

开发周期：3 个月

生命周期模型：敏捷迭代模型（Scrum）

一、文档阅读心得：从“建立规范”到“持续改进”

通过详细阅读课程群中发布的三个核心国际标准文档，我对软件过程管理有了立体化的认知，它们共同构成了一个“定义-实施-评估-改进”的完整闭环：

- ISO/IEC 12207（软件生命周期过程）**：主要解决“做什么”的问题。它像一本全面的字典，将软件从“受孕”到“退役”的整个生命周期划分为系统化的过程（如获取、供应、开发、运行、维护），并向下细化为活动和任务。它让我认识到，写代码仅仅是“软件实现过程”中的一小部分，一个健壮的软件离不开外围的支持和管理过程。
- ISO/IEC 15504 (SPICE) 及 ISO/IEC 33001**：主要聚焦解决“做得怎么样”的问题。15504 提供了软件过程评估的框架，而 33001 是其演进版本。它们引入了过程能力级别（如从 Level 0 不完整级到 Level 5 创新级）的概念。这让我明白，定义了过程还不够，团队还需要一套科学的标尺去度量当前过程的执行能力，从而找到瓶颈并进行持续的过程改进（SPI）。

二、项目软件过程的定义与细化

结合我的项目实践，基于 ISO/IEC 12207 的框架体系，我将本项目的软件过程定义为以下三大过程组，并明确了核心过程的输入与输出：

1. 主要生命周期过程 (Primary Processes)

(1) 需求分析与工程过程：

活动：收集校内用户痛点，竞品分析，梳理用户故事（User Story），绘制原型图。

关键产物：《产品需求文档 (PRD)》（轻量级）、Axure/墨刀原型图。

(2) 架构与设计过程：

活动：确定前后端技术栈（微信原生 + Spring Boot），设计数据库表结构，定义前后端交互 API。

关键产物：数据库 ER 图、Apifox 在线接口文档。

(3) 软件实现（编码）过程：

活动：前后端分离编码，组件封装，代码注释。

关键产物：源代码（提交至代码仓库）。

(4) 软件测试过程：

活动： 开发者单元测试、前后端联调测试、系统功能验收测试、多机型兼容性测试。

关键产物： 测试用例清单、Bug 追踪面板。

2. 支持过程 (Supporting Processes)

- (1) **配置管理过程：** 采用 Git Flow 工作流，规定了 main（生产环境）、develop（开发环境）和 feature（特性开发）的分支管理策略，保障代码资产的安全与版本可追溯。
- (2) **质量保证过程：** 通过 GitHub/Gitee 上的 Pull Request (PR) 机制进行强制的同行代码审查（Code Review），审查通过后方可合并入主分支。
- (3) **问题解决过程：** 利用协作工具（如 Teambition）的看板，按“待处理-处理中-待验证-已关闭”的流转机制管理系统缺陷。

3. 组织与管理过程 (Organizational & Management Processes)

项目策划与控制过程： 每两周为一个 Sprint（冲刺），召开冲刺计划会确定两周的开发任务；每日进行 15 分钟站立会议同步进度和阻塞点。

三、 软件过程定义的来源与剪裁（Tailoring）策略

1. 过程定义的来源

本项目的过程定义主要来源于 **ISO/IEC 12207 标准** 的基础框架，同时深度融合了 **Scrum 敏捷开发框架** 的工程实践。标准提供了完备性视角，敏捷提供了执行层的灵活性。

2. 剪裁的想法与核心依据

ISO/IEC 12207 旨在涵盖包括航空航天、金融等高复杂度的系统，而我们的项目是**团队规模小、生命周期短、容错率相对较高的互联网应用**。根据“适度原则”与“成本效益”，如果生搬硬套国际标准，将导致管理成本远超开发成本。因此，我们进行了如下实质性的过程剪裁：

剪裁维度	ISO/IEC 12207 标准要求（重型）	本项目剪裁后的实践（轻量级）	剪裁理由与想法
过程裁减	包含获取、供应、维护、退役等全要素过程。	完全剔除协议过程（获取/供应）和退役过程。	校园自发项目无外部商业合同约束；项目交付即课程结束，暂无明确的长期维护和退役计划。

角色合并	设有独立的项目经理、架构师、配置管理员、质量保证工程师。	角色兼任，如开发兼任架构设计，产品兼任测试，全员参与质量审查。	团队仅有 5 人，资源极度有限，通过跨职能协作打破角色壁垒，提高沟通效率。
文档轻量化	要求详尽的《需求规格说明书》、《详细设计说明书》、《测试计划》等。	工具替代文档：使用原型图代需求文档，使用在线接口平台代设计文档，使用 Kanban 代进度报告。	遵循敏捷思想：“工作的软件高于详尽的文档”。降低文档维护成本，避免文档与代码脱节。
模型替换	偏向于传统的、阶段边界清晰的瀑布模型。	采用敏捷迭代模型，将设计、编码、测试融合在两周的 Sprint 中循环进行。	校园二手市场需求模糊且易变，迭代模型能更快地交付可用版本（MVP）以收集用户反馈。

总结：

通过这次课程学习和作业实践，我认识到软件过程管理并非死板的教条。标准不是用来限制开发的，而是提供一个最佳实践的资源库。优秀的软件过程管理，正是建立在深刻理解标准的基础上，结合项目自身的上下文（如团队能力、业务风险、时间要求）进行精准的“剪裁”，最终在“规范性”与“敏捷性”之间找到最适合当前项目的平衡点。